

## 15 — Un indice per cercare

**La domanda a cui risponde:** cercare fra 21.000 carte sparse in 189 file senza scaricarli tutti. Quando conviene costruire un **indice**, quanto può pesare prima di non convenire più, e perché un tetto sui risultati non basta se il costo vero sta altrove.

### Il fatto

Una carta fisica si identificava digitando il numero stampato in basso: 118/191. L'app filtrava i set che hanno **191 carte in tutto** e cercava la 118 in ognuno.

Funziona, ma è un identificatore debole per due motivi diversi.

**Il totale non identifica il set.** 165 è sia 151 che Expedition; 101 è cinque set. I candidati non arrivano perché somigliano alla tua carta: arrivano per **coincidenza aritmetica**.

**Le promo il totale non ce l'hanno.** Un Black Star Promo dice 032 e basta. Non esiste nessun numero da mettere nel secondo campo, e il form lo pretendeva:

```
if (!numero || !totale) return;
```

Risultato: oltre 750 promo **italiane, già presenti nei dati**, erano inaccessibili. Non per mancanza di dati — per mancanza di una domanda che l'app sapesse fare.

Il nome invece è scritto grande sulla carta. Ma da solo non basta: *Pikachu* esiste in **107 stampe**. È la coppia che funziona.

| chiave               | quante carte identifica              |
|----------------------|--------------------------------------|
| numero + totale      | i candidati arrivano per coincidenza |
| nome                 | <i>Pikachu</i> □ 107                 |
| <b>nome + numero</b> | <b>una sola nel 97% dei casi</b>     |

### Perché un indice

`cercaPerNome()` esisteva già, ma cerca **solo nei set caricati in memoria**, e il suo commento spiegava perché: caricarli tutti significa 8,6 MB a ogni ricerca.

Il punto da vedere è che **per cercare un nome non servono le carte**. Serve sapere *dove stanno*. È una struttura molto più piccola dei dati che indicizza:

```
{ "articuno ex": "np:32", "pikachu": "base1:58 sv08:57 ..." }
```

`tools/genera-indice-nomi.mjs` lo costruisce una volta, a casa, e il risultato si committa. La PWA lo scarica (257 KB) e poi apre **solo** i file dei set che compaiono fra i risultati.

È lo stesso mestiere di un indice di database — o dell'indice analitico in fondo a un libro. Chi viene da SQL riconoscerà anche il trucco: si accetta di **duplicare** un'informazione già presente nei dati, in cambio di un accesso che non richiede di scorrerli tutti. E come per un indice SQL, la duplicazione si paga quando i dati cambiano: qui, rilanciando lo strumento.

Una scelta di formato che vale la pena notare:

```
indice[nome] = posizioni.join(' '); // "base1:58 sv08:57"
```

invece di un array di oggetti. Sono gli stessi dati, ma il JSON quasi dimezza — e questo file lo scarica ogni telefono al primo avvio. Le strutture “belle” costano, e il posto dove si vede è il filo.

### La trappola: un tetto sulla cosa sbagliata

La ricerca parziale va limitata, altrimenti chi digita a fa corrispondere migliaia di nomi. Il primo tetto era ovvio:

```
const MAX_NOMI = 40; // quante voci dell'indice espandere
```

Sembrava ragionevole. Poi la prova sui dati veri:

"ar" → 425 risultati, 138 set, 140 file scaricati

Quaranta nomi, sì. Ma quei quaranta nomi sono **ristampati in 138 set diversi**, e ogni set è un file. La ricerca scaricava praticamente l'intero catalogo — cioè esattamente ciò da cui l'indice doveva proteggere.

L'errore è di ragionamento, non di codice: avevo messo il tetto sull'unità che stavo **guardando** (i nomi) invece che su quella che **costa** (i file). Fra le due non c'è nessuna proporzione: un nome può stare in un set o in ottanta.

```
const MAX_SET = 12; // ogni set è un file: il costo vero
const MAX_CANDIDATE = 60; // ogni carta è una riga da disegnare
```

Con questi: "ar" □ 16 risultati, 12 set, 14 file. Da megabyte a qualche decina di KB.

**La lezione generalizzabile:** quando limiti qualcosa, chiediti *qual è l'unità che pago*. Spesso non è quella che stai contando. Qui il rapporto fra le due era 1 a 3,5 — e solo la misura l'ha rivelato.

## Dire la verità sul troncamento

Un risultato troncato in silenzio è peggio di un errore: chi cerca vede quindici carte, non trova la sua, e conclude che non esista.

```
return { trovate, nonLetti, troppi };
```

E il consiglio cambia col caso, perché non è la stessa situazione:

numero

```
? 'Ci sono altre carte con questo nome: se non è fra queste, controlla il numero.'
: 'Troppe carte con questo nome: aggiungi il numero stampato sulla carta.'
```

A chi ha già scritto "Articuno" per intero, dire "scrivi un pezzo più lungo" non serve a niente: il nome è quello. Il rimedio è il numero. Un messaggio generico è un messaggio che non aiuta nessuno dei due casi.

## L'accoppiamento invisibile

Qui c'è la parte più insidiosa, e non si vede leggendo nessuno dei due file.

L'indice lo **scrive** `normalizzaNome` di `src/engine/nomi.js`. Lo **legge** la `normalizza` privata di `src/data/dataset.js` — una copia deliberata, perché `data/` non deve dipendere da `engine/` (il flusso va da `data` verso `engine`, non indietro).

Due funzioni identiche in due file diversi restano identiche **finché qualcuno non tocca una sola delle due**. E se divergono di un carattere, non succede niente di visibile: l'indice non dà errore, **smette di trovare** le carte il cui nome cade nella differenza. Un guasto che nessuno nota per mesi.

La difesa è un test che attraversa entrambe le strade con nomi scelti apposta per cadere sulle regole di normalizzazione:

```
const INSIDIOSI = ['Shaymin-V', 'Oscurità', 'Nidoran ', 'Mr. Mime', 'PIKACHU', ...];
// indice costruito con normalizzaNome (engine), interrogato con dataset (data)
```

Un progetto Java avrebbe risolto l'accoppiamento con un modulo condiviso e il compilatore a fare da guardia. Qui la duplicazione è una scelta di architettura, e il prezzo è che **la guardia te la scrivi tu**. Vale la pena notare che è un prezzo, non un pareggio.

Gli altri due test sono sui **dati veri**, non su fixture: ogni posizione dichiarata deve puntare a una carta che esiste, e l'indice deve coprire tutti i set presenti nel repository. Il secondo esiste perché rilanciare `genera-indice-nomi.mjs` dopo `scarica-set.mjs` è facilissimo da dimenticare, e dimenticarlo non rompe niente: semplicemente le carte nuove non si trovano.

## Il costo, detto per intero

`data/nomi.json` sta nel GUSCIO del service worker: 257 KB, il file più pesante del precaricamento, quasi il triplo di tutti gli altri indici messi insieme.

Non è gratis e non va raccontato come se lo fosse. Ci sta perché l'alternativa per cercare un nome offline è scaricare 8,6 MB di set, e perché senza di esso una categoria intera di carte — le promo — resta fuori dal catalogo.

## Identificare non è cercare

Aggiunta di luglio 2026, e viene da un bug segnalato così: «la ricerca per nome funziona solo sulle mie carte». Era vero due volte, e il secondo motivo è quello interessante.

Il primo: nella collezione le carte mancanti si appendevano alla sezione di ogni set **senza passare da nessun filtro**, quindi il testo digitato non le toccava. Un guasto silenzioso — filtrando per tipo Fuoco arrivavano comunque tutte le carte del set — che si è visto solo quando qualcuno ha cercato un nome.

Il secondo: la ricerca globale esisteva già, ma con una regola che qui era sbagliata. `cercaPerNomeGlobale()` dava la **precedenza al nome esatto**: se quello che hai scritto è un nome intero, gli omonimi più lunghi non si guardano nemmeno.

```
const chiavi = precedenzaEsatta && cacheNomi[ago]
  ? [ago] // solo "Pikachu"
  : Object.keys(cacheNomi).filter((k) => k.includes(ago)); // anche "Pikachu ex"
```

Quella regola è giusta per il mestiere per cui è nata: **identificare** la carta che hai in mano. Scrivi “Articuno”, e non vuoi vederti annegare fra “Articuno ex” e “Articuno V” — la carta è una, ed è quella.

Ma **cercare** è il mestiere opposto. Chi scrive “pikachu” nella casella della collezione vuole vedere tutti i Pikachu che gli mancano, ex e V compresi. Stessa funzione, stesso indice, intenzione capovolta.

Da qui la morale, che vale molto oltre questo caso: quando due chiamanti vogliono comportamenti diversi, la scelta va **al chiamante** — un parametro con un default che conserva il comportamento di prima — non a un `if` dentro la funzione che indovina chi sta chiamando. Il default che non cambia niente è la parte che rende la modifica sicura: nessuno dei chiamanti esistenti si accorge di nulla, e il test che sorvegliava la vecchia regola resta verde senza toccarlo.

In Java sarebbe un overload, o un `enum` di strategia. In JS il parametro oggetto destrutturato con default (`{ precedenzaEsatta = true } = {}`) fa lo stesso lavoro senza firme duplicate: si legge sul posto della chiamata, che è dove serve capirlo.

E i tetti? Restano quelli, con una conseguenza da conoscere: “Pikachu” esiste in 107 stampe, quindi i limiti scattano sul primo nome e le varianti non fanno in tempo a entrare. La ricerca lo dichiara — «Ci sono troppe carte con questo nome: queste sono le prime» — che è la stessa disciplina del §*Dire la verità sul troncamento*, applicata a una schermata diversa.

## Verifica

1. Il tetto era su `MAX_NOMI` e il costo stava nei file dei set. Nel resto del progetto, trova un altro punto in cui l'unità che si conta non è l'unità che si paga. (Suggerimento: guarda il caricamento delle immagini.)
2. L'indice mappa nome a posizioni. Progetta l'indice che servirebbe per cercare **per attacco** (“quali mie carte fanno almeno 100 danni?”). Quanto peserebbe, e cosa cambierebbe rispetto a questo?
3. `normalizza` è duplicata di proposito fra `data/` e `engine/`. Elenca due modi alternativi di evitare la divergenza senza creare la dipendenza che si vuole evitare, e di che cosa costa ciascuno.
4. Il campo `troppi` dice a chi chiama che la risposta è incompleta. Perché non basterebbe restituire semplicemente più risultati, o meno?
5. `precedenzaEsatta` ha `true` come default, cioè il comportamento di prima. Cosa sarebbe andato storto scegliendo `false`, visto che il chiamante nuovo vuole proprio `false`?

6. Cercando “pikachu” i tetti scattano sul nome più corto e le varianti restano fuori. Progetta un modo di distribuire i posti disponibili **fra** i nomi corrispondenti invece di esaurirli sul primo: cosa ci si guadagna, e cosa si perde per chi sta identificando una carta in mano?